Problem Statement:

There is a biker going on a road trip. The road trip consists of n + 1 points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array gain of length n where gain[i] is the **net gain in altitude** between points i and i + 1 for all (0 <= i < n). Return *the **highest altitude** of a point*.

Example 1:

Input: gain = [-5,1,5,0,-7]

Output: 1

Explanation: The altitudes are [0,-5,-4,1,1,-6]. The highest is 1.

Example 2:

Input: gain = [-4,-3,-2,-1,4,3,2]

Output: 0

Explanation: The altitudes are [0,-4,-7,-9,-10,-6,-3,-1]. The highest is 0.

Constraints:

- n == gain.length
- 1 <= n <= 100
- -100 <= gain[i] <= 100

Program:

```
class Solution {
  public int largestAltitude(int[] gain) {
    int maxAltitude = 0;   // starting altitude is 0
    int currentAltitude = 0;

    // Traverse through the gain array
    for (int g : gain) {
      currentAltitude += g;        // update altitude
      maxAltitude = Math.max(maxAltitude, currentAltitude); // track highest
    }

    return maxAltitude;
```

```
    }

    // Quick test
    public static void main(String[] args) {
        Solution sol = new Solution();

        int[] gain1 = {-5,1,5,0,-7};
        System.out.println(sol.largestAltitude(gain1)); // Output: 1

        int[] gain2 = {-4,-3,-2,-1,4,3,2};
        System.out.println(sol.largestAltitude(gain2)); // Output: 0
    }
}
```

1. Input: gain = [-5,1,5,0,-7]

Output: 1

2. Input: gain = [-4,-3,-2,-1,4,3,2]

Output: 0